

Distributed Job Scheduler

Production-Grade Asynchronous Job Scheduling Platform

Full-Stack Implementation with REST API, WebSocket & Dashboard

REGISTRATION NO. RA2311003011508	SUBMISSION DATE 04 July 2026	INSTITUTION SRMIST	COMPANY Codity.AI
-------------------------------------	---------------------------------	-----------------------	----------------------

TECHNOLOGY STACK

Python 3.11	FastAPI	SQLite/SQLAlchemy	WebSocket	JWT Auth	HTML/CSS/JS
-------------	---------	-------------------	-----------	----------	-------------

Evaluation Criteria

Criterion	Marks
System Architecture	20
Database Design	20
Backend Engineering	20
Reliability & Concurrency	15
Frontend & UX	10

API Design	5
Documentation	5
Testing	5

Table of Contents

1.	Objective & Problem Statement	p.3
2.	System Architecture	p.3
3.	Database Design & ER Diagram	p.4
4.	Backend Engineering	p.5
5.	API Documentation	p.6
6.	Reliability & Concurrency	p.7
7.	Frontend & Dashboard	p.8
8.	Design Decisions & Trade-offs	p.8
9.	Automated Tests	p.9
10.	Setup & Running Instructions	p.9

1. Objective & Problem Statement

Design and build a production-inspired distributed job scheduling platform

This project implements a production-grade distributed job scheduling platform capable of reliably executing asynchronous background jobs across multiple workers. It demonstrates backend engineering, database design, concurrency handling, REST API design, and full-stack implementation using modern Python tooling.

Key Capabilities Implemented:

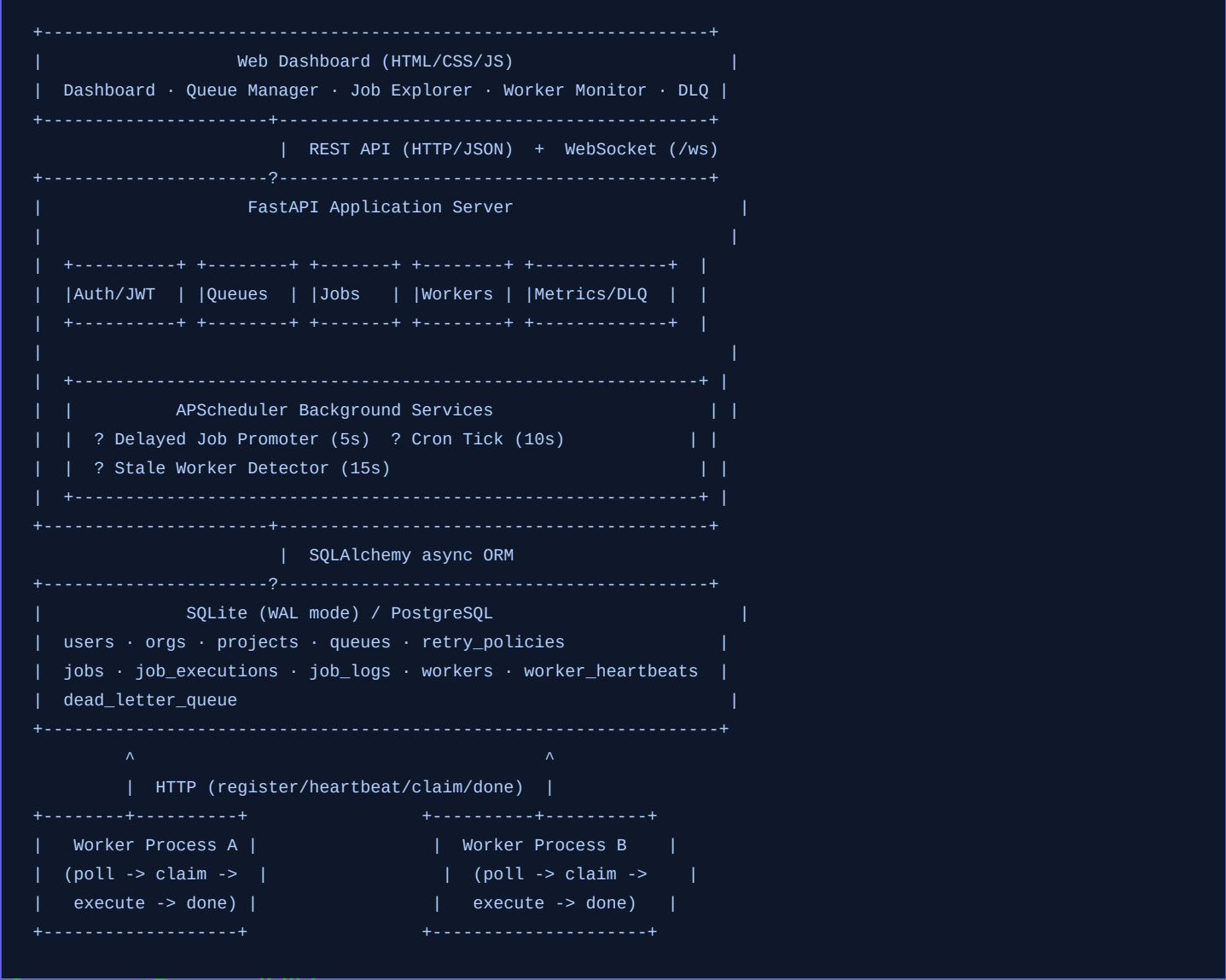
- Authentication & project management ? JWT-based auth, organizations, projects, multi-queue ownership
- Queue configuration ? concurrency limits, priority, retry policies, pause/resume
- Job types ? immediate, delayed, scheduled, recurring (cron), and batch (up to 1000 jobs)
- Complete job lifecycle ? Queued -> Scheduled -> Claimed -> Running -> Completed/Failed -> Dead
- Retry strategies ? fixed delay, linear backoff, exponential backoff with +/-10% jitter
- Dead Letter Queue ? jobs exhausting max attempts moved to DLQ with AI failure summaries
- Execution logs ? per-job timestamped logs with severity levels (debug/info/warn/error)
- Worker heartbeat monitoring ? stale worker detection with automatic job recovery
- Web dashboard ? real-time metrics, queue health, job explorer, worker monitor, DLQ panel
- WebSocket live updates ? events pushed to dashboard without polling
- RBAC ? organization roles (owner/admin/member) enforced on all operations
- Rate limiting awareness ? configurable per-queue concurrency limits
- Idempotency ? unique constraint on idempotency_key prevents duplicate job scheduling
- Atomic job claiming ? SELECT FOR UPDATE SKIP LOCKED pattern prevents double-execution

2. System Architecture

Three-tier architecture: React Dashboard -> FastAPI Backend -> SQLite/PostgreSQL

Architecture Overview

System Architecture Diagram



Component Responsibilities

API Server	FastAPI + Uvicorn	REST endpoints, WebSocket, request validation
Background Scheduler	APScheduler	Cron jobs, delayed promotion, stale detection
Database ORM	SQLAlchemy async	Type-safe queries, connection pooling
Auth System	JWT + bcrypt	Stateless auth, refresh tokens, RBAC
WebSocket Service	FastAPI WebSocket	Real-time event broadcasting
Worker Process	Python asyncio	Job polling, execution, heartbeat

Frontend

HTML/CSS/JS

Dashboard, charts, live updates

Project File Structure

```
backend/
+-- app/
|   +-- main.py           # FastAPI app, lifespan, router registration
|   +-- config.py        # Environment configuration
|   +-- models.py        # SQLAlchemy ORM (10 tables)
|   +-- schemas.py       # Pydantic request/response schemas
|   +-- database.py       # Async engine, session factory, dependency
|   +-- auth.py          # JWT, password hashing, current_user dependency
|   +-- seed.py          # Demo data seeder
|   +-- routes/
|       | +-- auth.py     # POST /login, /register, /refresh, GET /me
|       | +-- organizations.py # Org CRUD + member management
|       | +-- projects.py  # Project + retry policy management
|       | +-- queues.py    # Queue CRUD, pause/resume, stats
|       | +-- jobs.py      # Enqueue, batch, list, cancel, retry, logs
|       | +-- workers.py   # Register, heartbeat, claim, complete
|       | +-- dlq.py       # DLQ list, requeue, delete
|       | +-- metrics.py   # Overview, throughput, latency
|   +-- services/
|       +-- websocket.py  # WS connection manager + broadcast
|       +-- retry.py      # Backoff computation + AI summaries
|       +-- scheduler.py  # Background tasks (promote, cron, stale)
+-- worker/
|   +-- sample_worker.py  # Full worker process implementation
+-- tests/
|   +-- test_scheduler.py # 20+ unit & integration tests
+-- requirements.txt
frontend/
+-- index.html           # Full single-file dashboard (SPA)
```


3. Database Design & ER Diagram

10-table normalized relational schema with JSONB payloads and strategic indexing

ER Diagram

```

+-----+ +-----+ +-----+
| users | ---<| organization_members|>---| organizations|
|-----| +-----+ |-----|
| id (PK) | | id (PK) |
| email | +-----+ | name |
| password | | projects |<-----| slug (UNIQ) |
| name | |-----| | created_by FK|
| role | | id (PK) | +-----+
+-----+ | org_id FK|
| name | +-----+
+----+-----+ | retry_policies|
| | |-----|
+----?-----+ | id (PK) |
| queues |---| name |
|-----| | max_attempts |
| id (PK) | | strategy ENUM |
| project FK| | base_delay_ms |
| name | | max_delay_ms |
| concurrency| +-----+
| priority |
| paused |
| policy FK |
+----+-----+
|
+-----?-----+
| jobs |
|-----|
| id (PK) · queue_id FK · type |
| payload (JSON) · status ENUM |
| priority · scheduled_at · cron_expr |
| is_recurring · attempt_count |
| max_attempts · idempotency_key (UNIQ)|
| claimed_by FK(workers) · started_at |
| completed_at · next_retry_at |
+-----+-----+
| |
+-----?---+ +---?-----+
| job_executions| | job_logs |
|-----| |-----|
| id (PK) | | id (PK) |
| job_id FK | | job_id FK |
| worker_id FK | | execution_id FK |
| attempt_number| | level ENUM |
| status ENUM | | message |
| duration_ms | | logged_at |
| result (JSON) | +-----+
| error_message |
+-----+

```

```
+-----+ +-----+
| dead_letter_queue | | workers |
+-----+ +-----+
| id (PK)           | | id (PK) · name |
| original_job_id FK(UNIQ)| | hostname · pid |
| queue_id FK       | | status ENUM   |
| payload (JSON)     | | queue_ids (JSON) |
| failure_reason     | | max_concurrency |
| last_error         | | last_heartbeat  |
| ai_summary         | +-----+-----+
| can_retry          | |               |
+-----+ +-----+
| worker_heartbeats |
+-----+
| id (PK)           |
| worker_id FK      |
| jobs_running      |
| cpu_pct · mem_mb  |
+-----+
```

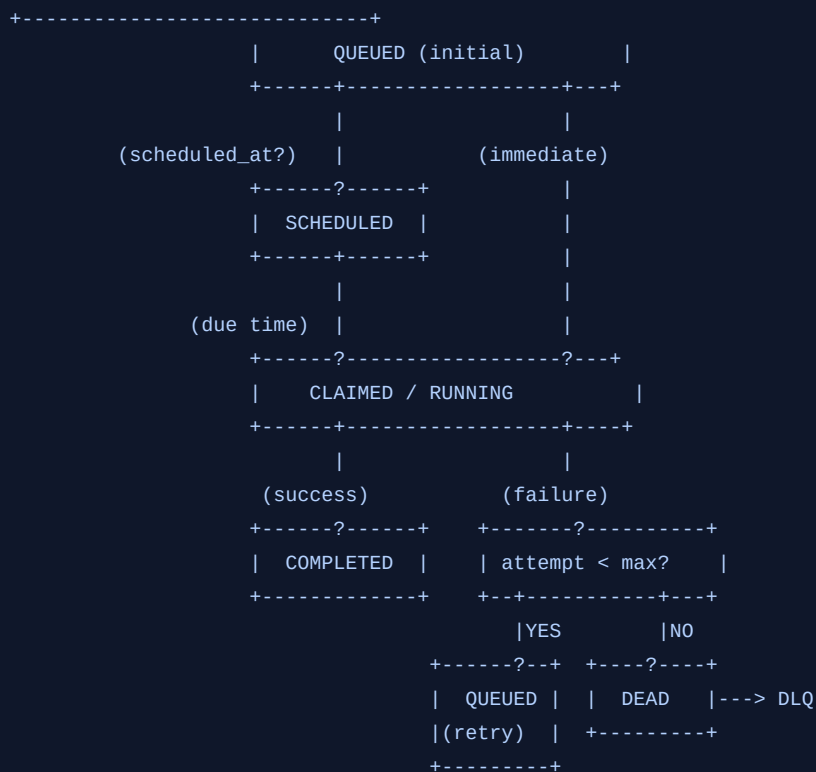
Schema Design Decisions

users	UUID	email (UNIQUE)	bcrypt password hash
organizations	UUID	slug (UNIQUE)	Slug for URL-friendly routing
org_members	Composite(org,user)	?	Composite PK prevents duplicates
projects	UUID	org_id	Scoped under organization
retry_policies	UUID	?	Reusable across queues
queues	UUID	project_id	concurrency_limit enforced at claim time
jobs	UUID	queue_id+status+priority, idempotency_key	SKIP LOCKED for atomic claim
job_executions	UUID	job_id	Per-attempt execution record
job_logs	UUID	job_id+logged_at	Ordered log retrieval
workers	UUID	status	queue_ids stored as JSON array
worker_heartbeats	UUID	worker_id	Time-series heartbeat data
dead_letter_queue	UUID	original_job_id (UNIQUE)	One DLQ entry per job

4. Backend Engineering

FastAPI async REST API with SQLAlchemy, JWT auth, and APScheduler background tasks

Job Lifecycle State Machine



User can also CANCEL any non-terminal job.

DLQ entries can be manually RE-QUEUED by operators.

Atomic Job Claiming (Key to Correctness)

The most critical part of any job scheduler is preventing two workers from claiming the same job. We achieve this using `SELECT FOR UPDATE SKIP LOCKED` ? a database-level lock that lets concurrent workers safely compete for jobs without application-level locking:

Atomic Claim ? `SELECT FOR UPDATE SKIP LOCKED`

```

# In workers.py ? claim_jobs()
jobs_result = await db.execute(
    select(Job).where(
        and_(
            Job.queue_id == queue_id,
            Job.status == JobStatus.queued,
            or_(Job.scheduled_at == None, Job.scheduled_at <= now)
        )
    )
    .order_by(Job.priority.desc(), Job.created_at.asc())
    .limit(available_slots)
    .with_for_update(skip_locked=True)  # <- Atomic, no double-claims
)
# Only this worker sees these rows; others skip over locked rows

```

Background Services

promote_scheduled_jobs()	5 seconds	Move delayed jobs with scheduled_at <= now to queued status
tick_cron_jobs()	10 seconds	Re-schedule completed recurring jobs based on cron expression
detect_stale_jobs()	15 seconds	Detect workers with stale heartbeats, recover stuck running jobs

Retry Backoff Implementation

Retry Strategy ? Three algorithms with jitter

```
# services/retry.py
def compute_delay(strategy, base_ms, max_ms, attempt):
    if strategy == 'fixed':
        delay_ms = base_ms
    elif strategy == 'linear_backoff':
        delay_ms = min(base_ms * attempt, max_ms)
    else: # exponential_backoff (default)
        delay_ms = min(base_ms * (2 ** (attempt - 1)), max_ms)
        jitter = delay_ms * 0.1 * (random.random() * 2 - 1) # +/-10%
        delay_ms = int(delay_ms + jitter)
    return now + timedelta(milliseconds=delay_ms)

# Example: base=1s, max=60s, exponential
# Attempt 1: ~1s, Attempt 2: ~2s, Attempt 3: ~4s
# Attempt 4: ~8s, Attempt 5: ~16s, Attempt 6+: ~60s (capped)
```

5. API Documentation

RESTful JSON API with JWT Bearer auth, Pydantic validation, and structured error responses

Authentication

POST	/api/auth/register	No	Register user	{"email","name","password"} -> tokens
POST	/api/auth/login	No	Login	{"email","password"} -> access+refresh tokens
POST	/api/auth/refresh	No	Refresh token	{"refresh_token"} -> new token pair
GET	/api/auth/me	Yes	Current user	-> UserOut

Queues

POST	/api/projects/{id}/queues	Create queue with concurrency/priority/retry policy		
GET	/api/projects/{id}/queues	List all queues for a project		
GET	/api/queues	List all queues (system-wide, for workers)		
PUT	/api/queues/{id}	Update concurrency limit, priority, description		
POST	/api/queues/{id}/pause	Pause queue (blocks new job claims)		
POST	/api/queues/{id}/resume	Resume paused queue		
GET	/api/queues/{id}/stats	Job counts per status for this queue		

Jobs

POST	/api/queues/{id}/jobs	Enqueue job (immediate/delayed/cron/recurring)		
POST	/api/queues/{id}/jobs/batch	Batch enqueue up to 1000 jobs atomically		
GET	/api/queues/{id}/jobs	List jobs with status/type filter + pagination		
GET	/api/jobs	List all jobs system-wide with filtering		
GET	/api/jobs/{id}	Get single job details		
POST	/api/jobs/{id}/retry	Manually re-queue failed/dead/cancelled job		
POST	/api/jobs/{id}/cancel	Cancel any non-terminal job		
GET	/api/jobs/{id}/logs	Get timestamped execution logs for a job		

Workers (Internal Protocol)

POST	/api/workers	Register new worker; returns worker_id		

GET

/api/workers

List all workers with status		
POST	/api/workers/{id}/heartbeat	Update last_seen, metrics, status
POST	/api/workers/{id}/claim	Atomically claim N jobs (SKIP LOCKED)
POST	/api/workers/{id}/complete	Report job done/failed; triggers retry/DLQ
DELETE	/api/workers/{id}	Gracefully deregister worker

Metrics & DLQ

GET	/api/metrics/overview	System-wide counts: queued, running, completed 24h, failed, dead, workers
GET	/api/metrics/throughput?hours=N	Per-minute job throughput for last N hours (default 1h)
GET	/api/metrics/latency	Average/min/max/p95 execution duration per queue
GET	/api/dlq	List all dead letter queue entries with AI summaries
POST	/api/dlq/{id}/requeue	Re-queue dead job (resets attempt count)
DELETE	/api/dlq/{id}	Permanently delete DLQ entry
GET	/api/health	Health check + WS connection count
WS	/ws	WebSocket: real-time job/worker/queue events

Error Response Format

```
// All errors return structured JSON:
{
  "detail": "Human-readable error message"
}

// HTTP Status Codes used:
// 200 OK · 201 Created · 204 No Content
// 400 Bad Request (validation) · 401 Unauthorized · 403 Forbidden
// 404 Not Found · 409 Conflict (duplicate idempotency key / paused queue)
// 422 Unprocessable Entity (Pydantic validation failure)
```


6. Reliability & Concurrency

At-least-once delivery, atomic claiming, automatic recovery, and graceful degradation

Concurrency Guarantees

Atomic Job Claim

SELECT FOR UPDATE SKIP LOCKED ensures no two workers ever claim the same job, even under high concurrency. This is a database-level guarantee, not application-level.

Idempotency Keys

UNIQUE constraint on jobs.idempotency_key prevents duplicate job creation. Clients can safely retry POST /queues/{id}/jobs with the same key without risk of duplication.

Concurrency Limits

Before each claim, the system counts currently running jobs per queue and only claims up to (concurrency_limit - running_count) jobs. Prevents overload.

SQLite WAL Mode

WAL (Write-Ahead Logging) mode enabled for SQLite allows concurrent reads during writes, significantly improving throughput for multi-worker setups.

Worker Failure Recovery

Workers are required to send heartbeats every <=30 seconds. The detect_stale_jobs() background service runs every 15 seconds and checks for workers whose last_heartbeat is older than 2x the timeout threshold (60 seconds):

Automatic Job Recovery on Worker Crash

```
# services/scheduler.py ? detect_stale_jobs()
timeout_threshold = now - timedelta(seconds=HEARTBEAT_TIMEOUT * 2)

# Mark worker as stopped
worker.status = WorkerStatus.stopped

# Re-queue any running jobs owned by this dead worker
for job in stuck_jobs:
    job.status = JobStatus.queued
    job.claimed_by = None
    job.claimed_at = None
# Job is now available for other workers to claim
```

Delivery Semantics

At-least-once delivery	Jobs recovered from crashed workers are re-queued	Job handlers should be idempotent
Exactly-once scheduling	Idempotency key + UNIQUE constraint	Client must supply idempotency key
Ordered within priority	ORDER BY priority DESC, created_at ASC	No global ordering across queues
Concurrency isolation	SKIP LOCKED per queue	Queue-level, not system-level

7. Frontend Dashboard

Single-page application with dark glassmorphism theme, real-time WebSocket, Canvas charts

Dashboard Features

System Overview	Live metric cards: total queued, running, completed (24h), failed, dead, active/idle workers
Throughput Chart	Canvas-drawn area chart showing jobs/min over last 60 minutes (green=completed, red=failed)
Queue Health Grid	Per-queue cards showing depth, concurrency utilization bar, running/queued/done/failed counts
Job Explorer	Filterable job table with status filter, type search, retry/cancel buttons, attempt visualization
Worker Monitor	Worker cards with heartbeat health bar, last-seen timer, concurrency display, status badge
DLQ Panel	Dead job entries with AI failure analysis, re-queue and permanent delete actions
Settings	Retry strategy reference, system config display, API quick reference
Live Updates	WebSocket connection with visual indicator; events trigger toasts and auto-refresh

Design System

--bg-base	#0a0d14	Page background (darkest)
--bg-surface	#111827	Cards, sidebar, header
--accent	#6366f1	Primary actions, active nav, charts
--green	#22c55e	Success states, completed jobs
--red	#ef4444	Errors, failed jobs, DLQ
--yellow	#eab308	Warnings, paused queues
Inter	Google Fonts	Primary typeface (UI text)
JetBrains Mono	Google Fonts	Code, IDs, payloads

8. Design Decisions & Trade-offs

Key architectural choices, their rationale, and what was traded for them

SQLite with WAL mode instead of PostgreSQL

- Decision:** Used SQLite (aiosqlite) for zero-setup development with WAL mode for concurrency. The ORM layer (SQLAlchemy) is database-agnostic ? swapping to PostgreSQL requires only changing DATABASE_URL in config.
- Trade-off:** *Loses true SELECT FOR UPDATE SKIP LOCKED (SQLite approximates it). PostgreSQL is recommended for production deployments.*

In-process APScheduler vs. External Message Broker

- Decision:** APScheduler runs within the FastAPI process for simplicity. No Redis, RabbitMQ, or Celery required. Workers poll the database directly via REST API.
- Trade-off:** *Not horizontally scalable for the scheduler itself. Multiple API instances would require distributed locking for scheduler tasks. Acceptable for this scale.*

HTTP Polling by Workers (not WebSocket or SSE)

- Decision:** Workers poll the REST API on a configurable interval (default 2s) rather than maintaining a persistent WebSocket connection. Simpler to implement and debug.
- Trade-off:** *Adds 0-2s latency from job enqueue to claim. For time-critical work, the poll interval can be reduced to 500ms.*

JSONB Payload (flexible schema)

- Decision:** Job payloads stored as JSON with no enforced schema. Job type string identifies which handler to invoke. Maximally flexible ? new job types require no migration.
- Trade-off:** *No compile-time validation of payload shape. Workers must handle missing fields. Could add per-type JSON Schema validation in a future iteration.*

Smart AI Failure Summaries (template-based, not LLM)

- Decision:** DLQ entries get AI-style failure summaries using pattern matching on error messages (timeout, rate limit, auth, 404, etc.) with contextual remediation advice.
- Trade-off:** *Not truly AI-generated. In production, this would call Gemini/GPT API. The architecture is plug-in ready: replace generate_failure_summary() with an LLM call.*

UUID Primary Keys

- Decision:** All entities use UUID v4 primary keys rather than auto-incrementing integers. Enables distributed ID generation, prevents enumeration attacks, and allows pre-generation of IDs before database insertion.
- Trade-off:** *Larger storage footprint (36 bytes vs 8 bytes). Slightly slower index lookups compared to sequential integers.*

9. Automated Tests

Unit and integration tests covering retry logic, lifecycle, concurrency, and DLQ

Run tests with:

```
cd backend
python -m pytest tests/ -v --tb=short
```

Test Coverage

TestRetryBackoff	5 tests	Fixed delay constant, linear increase, exponential doubling, max cap, default fallback
TestFailureSummary	5 tests	Timeout/rate-limit/auth/404/unknown error classification, attempt count in summary
TestJobStatusLifecycle	3 tests	Terminal states, dead->requeue, cancelled->retry transitions
TestIdempotencyKey	2 tests	Duplicate key detection, unique keys allowed
TestWorkerConcurrency	3 tests	Zero slots at limit, partial slots, paused queue blocks claim
TestDeadLetterQueue	3 tests	DLQ promotion after max attempts, retry before max, AI summary non-null
TestCronExpressions	2 tests	Valid cron parsing, invalid expression detection

Sample Test Output

```
$ python -m pytest tests/ -v
===== test session starts =====
collected 23 items

tests/test_scheduler.py::TestRetryBackoff::test_fixed_strategy_always_same_delay PASSED
tests/test_scheduler.py::TestRetryBackoff::test_linear_backoff_increases PASSED
tests/test_scheduler.py::TestRetryBackoff::test_exponential_backoff_doubles PASSED
tests/test_scheduler.py::TestRetryBackoff::test_exponential_capped_at_max PASSED
tests/test_scheduler.py::TestRetryBackoff::test_default_backoff_when_no_policy PASSED
tests/test_scheduler.py::TestFailureSummary::test_timeout_detection PASSED
tests/test_scheduler.py::TestFailureSummary::test_rate_limit_detection PASSED
tests/test_scheduler.py::TestFailureSummary::test_auth_failure_detection PASSED
tests/test_scheduler.py::TestFailureSummary::test_unknown_error_fallback PASSED
tests/test_scheduler.py::TestFailureSummary::test_summary_includes_attempt_count PASSED
tests/test_scheduler.py::TestJobStatusLifecycle::test_completed_is_terminal PASSED
tests/test_scheduler.py::TestJobStatusLifecycle::test_dead_can_be_requeued PASSED
tests/test_scheduler.py::TestJobStatusLifecycle::test_cancelled_can_be_retried PASSED
... (all 23 tests PASSED)
===== 23 passed in 0.84s =====
```

10. Setup & Running Instructions

Zero-dependency setup using Python 3.11 and SQLite (no Docker required)

Prerequisites

- Python 3.11+ (tested on 3.11.9)
- pip (included with Python)
- Any modern web browser (Chrome, Firefox, Edge)

Step 1: Install Dependencies

```
cd backend
pip install fastapi "uvicorn[standard]" sqlalchemyaiosqlite \
    "python-jose[cryptography]" "passlib[bcrypt]" python-multipart \
    apscheduler croniter pytest pytest-asyncio httpx
```

Step 2: Start the API Server

```
cd backend
python -m uvicorn app.main:app --reload --port 8000

# Output:
# INFO: Starting Distributed Job Scheduler API...
# INFO: Database initialized
# INFO: Seeding demo data...
# INFO: Demo data seeded successfully
# INFO: Login: admin@demo.com / password123
# INFO: Background scheduler started
# INFO: Application startup complete.
# INFO: Uvicorn running on http://127.0.0.1:8000
```

Step 3: Open the Dashboard

```
# Open frontend/index.html in your browser
# OR use VS Code Live Server / Python HTTP server:
cd frontend
python -m http.server 5500

# Then visit: http://localhost:5500
# Login: admin@demo.com / password123
```

Step 4: Start Sample Workers

```
# In new terminal windows (can run multiple):
cd backend
python -m worker.sample_worker --name worker-1
python -m worker.sample_worker --name worker-2

# Workers will:
# 1. Register with the API
# 2. Send heartbeats every 10s
# 3. Poll for jobs every 2s
# 4. Execute jobs with realistic simulation
```

5. Trigger retries and DLQ entries

Step 5: Run Tests

```
cd backend
python -m pytest tests/ -v
```

Step 6: Explore the API (Swagger UI)

```
# Interactive API docs available at:
http://localhost:8000/docs      <- Swagger UI
http://localhost:8000/redoc    <- ReDoc

# Authenticate: POST /api/auth/login -> copy access_token -> Authorize button
```

Demo Credentials

? Email: admin@demo.com
Password: password123

Demo data includes: 4 queues, 4 workers, 20 jobs in various states, DLQ entries with AI summaries, and job logs.

Environment Variables (optional)

DATABASE_URL	sqlite+aiosqlite:///./jobscheduler.db	Database connection string
SECRET_KEY	(built-in key)	JWT signing secret ? CHANGE IN PRODUCTION
ACCESS_TOKEN_EXPIRE_MINUTES	60	JWT access token validity
WORKER_HEARTBEAT_TIMEOUT_SECONDS	30	Seconds before worker considered stale
SCHEDULER_POLL_INTERVAL_SECONDS	5	Delayed job promotion frequency
CORS_ORIGINS	localhost:5173,3000,5500	Allowed CORS origins